

Arquitetura Tecnica

Stack Recomendada

- Painel web: Next.js com TypeScript.
- App mobile: Expo/React Native com TypeScript.
- Back-end: NestJS ou Fastify com TypeScript.
- Banco: PostgreSQL.
- ORM: Prisma.
- Arquivos: S3 compativel.
- E-mails: Resend, SendGrid ou Amazon SES.
- Mapas: Google Maps Platform ou Mapbox.
- IA: OpenAI API.
- Autenticacao: JWT + refresh token, com RBAC por empresa.

Principio Central

Toda tabela operacional deve carregar `tenant_id` ou equivalente. Isso evita mistura de dados entre empresas e permite whitelabel real.

A ICEMAX deve ser tratada como tenant piloto, nao como regra fixa do sistema. O mesmo codigo devera atender ICEMAX, uma segunda empresa de ar-condicionado ou outro prestador tecnico com configuracoes proprias.

Aplicacoes

Painel Web

Responsavel por:

- Administracao da empresa.
- Usuarios e permissoes.
- Clientes.
- Equipamentos.
- Agenda.
- Ordens de servico.
- Estoque.
- Dashboard.
- Configuracoes whitelabel.

App Mobile

Responsavel por:

- Lista de OS atribuidas.
- Execucao do atendimento.
- Fotos.
- Checklist.
- Pecas usadas.
- Localizacao.
- Assinatura.
- Sincronizacao offline.

API

Responsavel por:

- Autenticacao.
- Regras de negocio.
- Controle multiempresa.
- Geração de PDF.
- Envio de e-mail.
- Integracoes com mapas.
- Integracoes de IA.

Offline

O app tecnico deve armazenar localmente:

- OS atribuidas.
- Dados essenciais do cliente.
- Dados do equipamento.
- Checklist.
- Fotos pendentes.
- Assinatura pendente.
- Eventos de atendimento.

Quando voltar a conexao, sincroniza eventos para a API. Conflitos devem priorizar registros com auditoria, nao sobrescrever silenciosamente.

Seguranca

- Senhas com hash forte.
- Tokens curtos com refresh token.
- Permissoes por papel e por empresa.
- Auditoria para mudancas sensiveis.
- Separacao logica por `tenant_id`.
- URLs assinadas para fotos e PDFs.
- Logs sem dados sensiveis desnecessarios.